Optimizing Triangle Strips for Fast Rendering

Francine Evans Steven Skiena Amitabh Varshney

State University of New York at Stony Brook

Abstract

Almost all scientific visualization involving surfaces is currently done via triangles. The speed at which such triangulated surfaces can be displayed is crucial to interactive visualization and is bounded by the rate at which triangulated data can be sent to the graphics subsystem for rendering. Partitioning polygonal models into triangle strips can significantly reduce rendering times over transmitting each triangle individually.

In this paper, we present new and efficient algorithms for constructing triangle strips from partially triangulated models, and experimental results showing these strips are on average 15% better than those from previous codes. Further, we study the impact of larger buffer sizes and various queuing disciplines on the effectiveness of triangle strips.

1 Introduction

Interactive display rates are crucial to exploratory scientific visualization and virtual reality. The speed of high-performance rendering engines on triangular meshes in computer graphics can be bounded by the rate at which triangulation data is sent into the machine. Obviously, each triangle can be specified by three vertices, but to maximize the use of the available data bandwidth, it is desirable to order the triangles so that consecutive triangles share an edge. Using such an ordering, only the incremental change of one vertex per triangle need be specified, potentially reducing the rendering time by a factor of three by avoiding redundant lighting and transformation computations. Besides, such an approach also has obvious benefits in compression for storing and transmitting models.

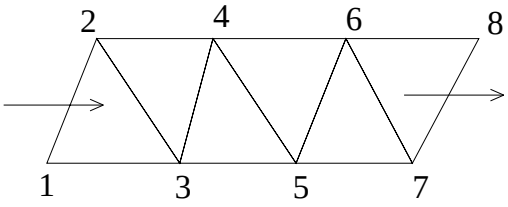


Figure 1: A Triangle Strip

Consider the triangulation in Figure 1. Without using triangle strips, we would have to specify the six triangles with three vertices each. By using triangle strips, as supported by the OpenGL graphics library [11, 12], we can describe the triangulation using the strip $(1, 2, 3, 4, 5, 6, 7, 8)$, and assuming the convention that the ith triangle is described by the ith, $(i + 1)$st, and $(i + 2)$nd vertices of the *sequential* strip. Such a sequential strip can reduce the cost to transmit n triangles from $3n$ to $n + 2$ vertices.

In this paper, we consider the problem of constructing good triangle strips from polygonal models. Often such models are not fully triangulated, and contain quadrilaterals and other non-triangular faces, which must be triangulated prior to rendering. The choice of triangulation can significantly impact the cost of the resulting strips. For example, Figure 2 demonstrates that one triangle strip suffices to represent a cube, provided it is triangulated in a particular manner. Although we have shown that the problem of triangulating a polygonal model for optimal strips is NP-complete [7], here we provide heuristics which exploit the freedom to triangulate these faces to produce strips that are on average 15% better than those of previous codes. Our linear-time algorithm manages to achieve this by exploiting both the local and the global structure of the model. Our analysis of the global structure of a geometric model is done via a non-geometric technique we term *patchification*, which we believe is of general interest as an efficient tool for logically partitioning polygonal models.

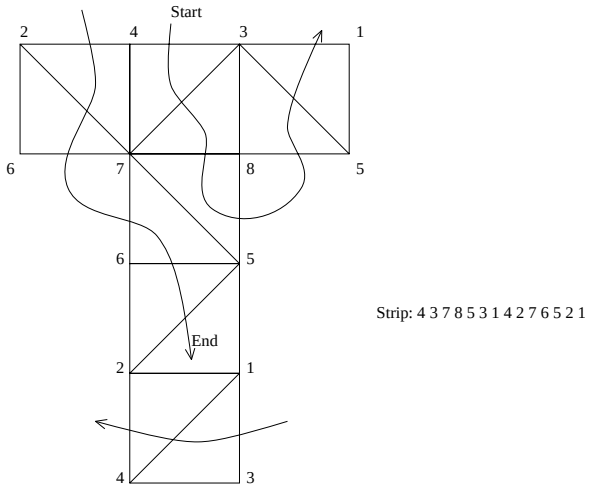


Figure 2: Triangulating a cube for one sequential strip.

To allow greater freedom in the creation of triangle strips, a "swap" command permits one to alter the FIFO (first-in, first-out) queuing discipline in a triangle strip [13]. A swap command swaps the order of the two latest vertices in the buffer so that instead of vertex i replacing the vertex $(i - 2)$ in a buffer of size 2, vertex i replaces the vertex $(i - 1)$. This allows for a single triangle strip representation of the collection of triangles shown in Figure 3, as $(1, 2, 3, SWAP, 4, 5, 6)$. This form of a triangle strip that includes swap commands is referred to as a *generalized triangle strip*.

The swap command gives greater freedom in the creation of triangle strips at the cost of one bit per vertex. Although the swap command is supported in the GL graphics library [13], keeping portability considerations in mind it was decided to not support it in OpenGL [8]. With OpenGL gaining rapid acceptance in the graphics software community, the one-bit-per-vertex cost model that was appropriate for a swap command in GL is now outdated. A more appropriate cost for such a swap command under the OpenGL model is a penalty of one vertex as explained next. One can simulate a swap command in OpenGL by re-transmitting the vertex that had

to be swapped. This results in an empty triangle two of whose vertices are the same. This is illustrated in Figure 3, where we simulate $(1, 2, 3, SWAP, 4, 5, 6)$ by $(1, 2, 3, 2, 4, 5, 6)$. Note that, even though a swap costs one vertex in the OpenGL model, it is still cheaper than starting a new triangle strip that costs two vertices. In this paper, we evaluate all algorithms for both the GL and OpenGL cost models.

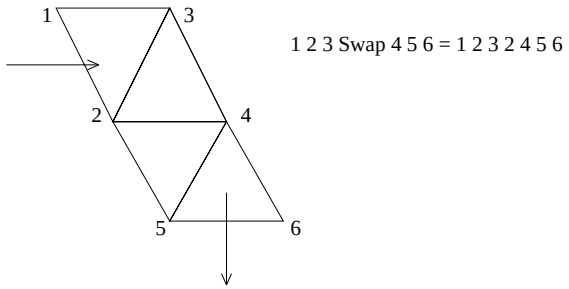


Figure 3: Replacing a swap requires an extra vertex.

Special-purpose rendering hardware is needed to fully exploit the advantages of triangle strips, by maintaining a buffer with the k previously transmitted vertices as determined by a certain queuing discipline. Although current rendering engines use a buffer of size of $k = 2$ and FIFO queuing discipline, there has been recent interest in studying the impact of larger buffer sizes, for both rendering [3] and geometric compression [6]. The decomposition of a triangular mesh into a triangle strip data structure that back-references the previous k vertices, $k \geq 2$ is referred to as a *generalized triangle mesh* [6]. Towards this end, we provide extensive analysis of the impact of buffer size and queuing discipline on triangle strip performance. We demonstrate that relatively small buffer sizes are sufficient to achieve most of the potential benefits of triangle strips, making for a desirable tradeoff between increasing hardware cost versus the speedup in rendering time.

In Section 2, we summarize previous work on triangular strips. In Section 3, we describe our local and global algorithms for constructing quality triangle strips from polygonal meshes. Experimental results are presented in Section 4. In Section 5, we study the impact of buffer size on triangle strip performance. Conclusions and plans for future work are discussed in Section 6.

2 Previous Work

The problem of constructing quality triangle strips has received attention from both the graphics and the computational geometry communities.

Akeley, Haeberli, and Burns have written a program that converts triangle meshes to triangle strips [1]. We discuss the approach in this program in greater details in Section 3. Deering has proposed the use of generalized triangle meshes for compressing connectivity information in geometric polygonal models [6]. He has proposed maintaining a stack of size $k = 16$ to store 16 previous vertices. A vertex for a new triangle is specified either through back-referencing one of the existing vertices on the stack, or by reading-in a new vertex and replacing an existing vertex on the stack. Although a novel idea, no algorithms have been proposed there to suggest how one can decompose polygonal models into generalized triangle meshes for a given buffer size k. An interesting alternative to compressing connectivity information is presented by Hoppe in [9] where vertex-split/edge-collapse information is encoded efficiently with respect to its neighbors. Although not as efficient as generalized triangle meshes for a single resolution model, this approach has the advantage of being able to encode multiresolution models compactly.

Within computational geometry, interest has focused on constructing and recognizing Hamiltonian and sequential triangulations. A triangulation is *Hamiltonian* if its dual graph contains a Hamiltonian cycle. Hamiltonian triangulations can be represented by using generalized triangle strips (triangle strips with swaps). Arkin, et.al. [2] proved that every point set has a Hamiltonian triangulation. Further, they showed that the problem of testing whether a triangulation is Hamiltonian is NP-complete. They gave an $O(n^2)$ algorithm for constructing a Hamiltonian triangulation of a polygon that has since been improved to $O(n \lg n)$ by Narasimhan [10].

A triangulation is *sequential* if its dual graph contains a Hamiltonian cycle whose turns alternate left-right. Sequential triangulations can be represented by using one triangle strip without any swaps. A Hamiltonian triangulation is sequential if three consecutive edges do not share a common vertex. Arkin, et.al. [2] proved that for any $n \geq 9$ there exists a set of n points in general position that do not admit a sequential triangulation. Although linear time suffices to test whether a triangulation is sequential, we [7] have shown that problem of finding a sequential triangulation of a partially triangulated surface is NP-complete using a reduction from 3-satisfiability. Hence, heuristics such as those described in this paper are required to find good sequential strips.

A simple path in the dual of a triangulation identifies a sequence of triangles that form a "strip" or a (triangular) "ribbon" . Bhattacharya and Rosenfeld [4] have studied geometric and topological properties of ribbons. The Hamiltonian triangulation problem can be considered that of identifying if a set of points or a polygon has a triangulation that consists of a single strip (triangular ribbon).

Bose and Toussaint [5] have recently studied a set of problems involving *quadrangulation* of point sets, and have obtained several interesting results. A quadrangulation of a point set S is a decomposition of the convex hull into quadrilaterals, such that each point of S is a vertex of some quadrilateral. In particular, they have applied the notion of Hamiltonian triangulations to this problem, and they have obtained an alternate method of computing Hamiltonian path triangulations.

By Euler's theorem on graphs, the number of triangles in a triangulation is at most twice the number of vertices, and on average we will have to send each vertex twice to the renderer using sequential triangle strips and a buffer of size 2. Bar-Yehuda and Gotsman [3] studied the extent to which we can increase the stack (buffer) size to reduce this duplication of vertices. This yields a time-versus-space tradeoff; for as we increase memory usage, rendering time will decrease. Bar-Yehuda and Gotsman have shown that a buffer of size $13.35\sqrt{n}$ is sufficient to render any mesh on n vertices in the optimal time n, and that a buffer size of $1.649\sqrt{n}$ is necessary for optimal rendering in the worst-case. They show the problem of minimizing the buffer-size for a given mesh is NP-hard, using a reduction from the problem of finding minimum separators of a planar graph.

3 Constructing Triangle Strips

In this section, we propose several heuristics for constructing triangle strips from polygonal models. There are at least three different objectives such heuristics might reasonably seek to achieve:

- *Maximize the length of each strip* – since each strip of length s represents $s - 2$ triangles, maximizing strip length minimizes this overhead.

- *Minimizing swaps* – since each swap costs one additional vertex in the OpenGL cost model.

- *Minimizing the number of singleton strips* – since each triangle left isolated after removing a strip creates a singleton strip, we should seek to begin and end our strips on low-degree faces of the triangulation.

The best previous code for constructing triangle strips which we are aware of is [1], implementing what we will call the SGI algorithm. The SGI algorithm seeks to create strips that tend to minimize leaving isolated triangles. It is a greedy algorithm, which always chooses as the next triangle in a strip the triangle that is adjacent to the least number of neighbors (i.e. minimizes the number of *adjacencies*). When there is more than one triangle with the same, least number of neighbors, the algorithm looks one level ahead to its neighbors' neighbors, and chooses the direction of minimum degree, choosing arbitrarily if there is again a tie. After starting from an arbitrary lowest degree triangle, it extends its strips in both directions, so that each strip is as long as possible. There is no reluctance to generate swaps, and understandably so, since this algorithm was aimed at generating triangle strips for Iris GL. A fast, linear-time implementation is obtained by using hash tables to store the adjacency information, linked to a priority queue maintaining strip length to choose which triangle starts a new strip.

Figure 4 illustrates how the algorithm breaks ties. Starting with a face of lowest adjacency (of degree 1 on the upper center of the figure), the algorithm always selects the lower degree face as the next triangle in the strip to peel off the marked strip. At the face of degree 3 it turns left because a neighbor to the left adjacent face is of degree 1 as opposed to 2.

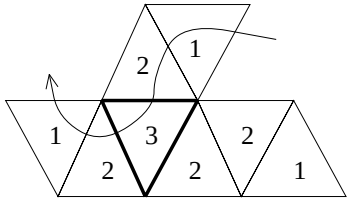


Figure 4: Adjacency counts in the SGI algorithm

The SGI algorithm uses strictly local adjacency information in constructing the triangle strips. However, fully exploiting the freedom to triangulate quads seems to require a more global approach. We have experimented with several variants of local and global algorithms, as discussed in the following two sections.

3.1 Local Algorithms

Our class of local heuristics starts from the same basic idea as the SGI algorithm – to use least adjacencies as the basis for choosing the next face in a strip. However, we have tried to improve upon their algorithm by dynamic triangulation and alternate tie-breaking procedures.

We have considered three different approaches to triangulating faces:

- *Static triangulation* – In this approach, we triangulate all quads and larger faces in our model as a pre-processing step before we begin finding strips. We use alternate left-right turns, as shown in Figure 5(b) because such a triangulation is inherently sequential, as opposed to the simpler and more conventional fan triangulation. The SGI algorithm accepts only triangulated models as input. Therefore, to compare their approach with ours we pre-triangulate all non-triangulated models using this static triangulation approach and then run their algorithm.

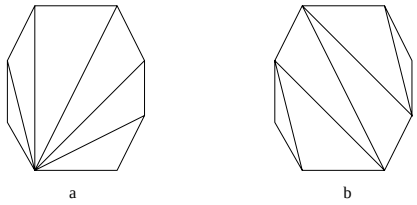


Figure 5: Fan vs sequential triangulation of a polygonal face.

- *Dynamic whole-face triangulation* – A second approach completely triangulates each face when we first enter it via some edge on a strip. After using one of the tie-breaking procedures described below to determine the exit edge e, we can triangulate the face as sequentially as possible while exiting at e. If the surface normals do not vary across a face, then whole face triangulation has the additional advantage of encoding fewer normal transitions.
- *Dynamic partial-face triangulation* – Partial-face triangulation provides the freedom to triangulate and walk only part of a face before exiting it. This approach can under certain conditions provably perform better than the whole-face triangulation, as is seen in the example where we represent a cube using a single sequential triangle strip. After identifying the exit edge e of the face with the minimum number of adjacencies, we sequentially triangulate the smallest portion possible of the face from the input edge to exit at e. This is illustrated in Figure 6.

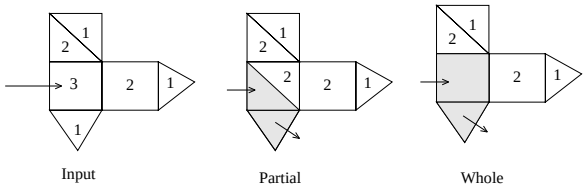


Figure 6: Examples of partial and whole-face triangulation.

We have considered several different approaches in breaking ties when there is more than one polygon that has the least number of adjacencies to the current face. Such ties often occur since the possible number of adjacencies ranges only over 1, 2, and 3. In particular, we tried:

- *Arbitrary* – meaning that we use the first face found among the low-adjacency faces.
- *Look-ahead* – this is the same approach that SGI algorithm takes, as described above.
- *Alternate* – this rule tries to alternate directions in choosing the next polygonal face. To motivate this option, note that sequential strips alternate directions.
- *Random* – chooses the next face randomly from those that were tied.
- *Sequential* – chooses the next face that will not produce a swap, and picks randomly if there is no such face.

To quickly identify the lowest adjacency face to start from, we maintain a priority queue ordered by the number of adjacent polygons to each face. The faces in the priority queue are linked to the adjacency list data structure representing the dual graph of the triangulation. This enables fast lookup to find and delete faces when forming the triangle strips.

3.2 Global Algorithms

Although the problem of finding the strip-minimal triangulation is NP-complete, we perform a global analysis of the structure of a polygonal model using a technique we call *patchification*, which we believe is of independent interest.

In typical polyhedral models, there are many quadrilateral faces, often arranged in large connected regions. We attempt to find large "patches", rectangular regions consisting only of quadrilaterals, as illustrated in Figure 7. Figure 8 shows the largest patches in a typical model. These patches can be triangulated sequentially along each row or column, although there is a cost of either 3 swaps per turn or 2 vertices to stop and restart each strip at the end of a row or column.

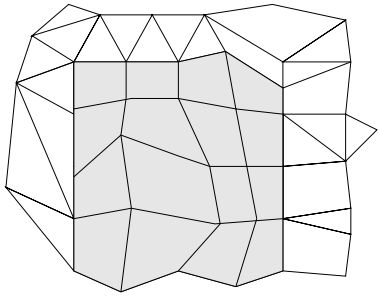


Figure 7: A rectangular patch of quadrilaterals.

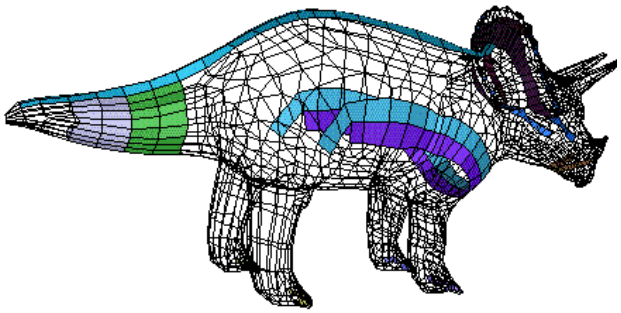


Figure 8: The six largest patches in a triceratops model.

Efficient patchification requires computing the number of polygons to the east, west, north, and south of each face, and making sure that when forming the patches, the polygons in the patch are all adjacent. Hence, we have to "walk" through the faces and calculate the number of adjacent polygons to them in each orientation. Each "walk" only visits each face exactly 2 times: once for the north-south direction and once for the east-west direction; once we visit a face in a walk, that face does not require visiting again. To avoid generating too many small patches, we keep a *patch cutoff size* which is the area of the smallest patch we would like to generate. Since we generate patches in decreasing order of size, we can conveniently stop the process once the areas of the patches being generated falls below this cutoff size. This approach takes us time $O(pn)$ where p is the number of patches found. In our studies p was much smaller than n and therefore this approach demonstrated a linear behavior.

We tried two different approaches for exploiting the coherence identified in large patches:

- *Row or column strips* – After selecting all patches whose size was greater than a specified cutoff size, we partitioned the patches into sequential strips along rows or columns (whichever direction yielded larger strips) and deleted them from the model. Next, a local algorithm (using whole-face triangulation) was used on the remaining model. By generating one strip along each row or column, we minimize the number of swaps needed.

- *Full-patch strips* – Each patch larger than the cutoff size was converted into one strip, at a cost of 3 swaps per turn. Further, every such strip was extended backwards from the starting quadrilateral and forwards from the ending quadrilateral of the patch to the extent possible. As before, the local algorithm was used on the model left after removing the patches and their forward and backward extensions.

4 Experimental Results

We have exhaustively tested our local and global algorithms on several datsets and compared them with the best known triangle strip code [1]. For our local approaches there were ten different options for each data file that we ran our experiments on: (a) whole-face triangulation and (b) partial-face triangulation, for each of the five tie breaking methods – (i) arbitrary, (ii) look-ahead, (iii) alternate, (iv) random, and (v) sequential. For our global approaches there were ten different options for each data file that we ran our experiments on: (a) row/column strips and (b) full-patch strips, for each of five different patch cutoff sizes of – 5, 10, 15, 20, and 25.

Table 1 shows the results of comparison of our best option, which was the global row/column strips with a patch cutoff size of 5, against the SGI algorithm. The cost columns show the total number of vertices required to represent the dataset in a generalized triangle strip representation under the OpenGL cost model (we are counting each vertex and swap that needs to be sent to the renderer).

Data File	Num Verts	Num Tris	Cost SGI	Cost Ours	Savings
plane	1508	2992	4005	3509	12%
skyscraper	2022	3692	5621	4616	18%
triceratops	2832	5660	8267	6911	16%
power lines	4091	8966	12147	10621	13%
porsche	5247	10425	14227	12367	11%
honda	7106	13594	16599	15075	9%
bell ranger	7105	14168	19941	16456	17%
dodge	8477	16646	20561	18515	10%
general	11361	22262	31652	27702	12%

Table 1: Comparison of triangle strip algorithms.

Figure 9 shows the performance comparisons between our best local and best global algorithms against the SGI algorithm for (a) GL and (b) OpenGL cost models. The models sorted by number of triangles are along the x-axis and the cost of generalized triangle strip representation is along the y-axis in this figure.

Observations include:

- Little if any savings seems possible by sophisticated algorithms under the GL model. However, under the more realistic model the combined local/global algorithm can save up to about 20% over the SGI algorithm.
- Our results are close to the theoretical lower bound of the number of triangles + (the number of connected components in the model * 2), so there is limited potential for better algorithms.
- Although the number of swaps required is sensitive to the composition of the model, the total cost grows linear in the size of the model.

Our times for execution of these algorithms behaved linearly with respect to the input size. The timings for our local algorithms were about a factor of two slower than those generated by SGI. Thus, for example, dynamic partial-face method with sequential triangulation took around 8 seconds on the $22K$ triangle model `general` whereas the SGI code took around 4 seconds.

For local algorithms under the GL cost model whole-face triangulations worked better than those with partial-face triangulations; under the OpenGL cost model the reverse was true. Partial-face triangulations produce less swaps than whole-face triangulations because the former have a greater choice in selecting the next face in a strip, and are therefore more likely to be able to select faces that do not require a swap. For global algorithms, full-patch strips with cutoff size of 25 have the best performance under the GL cost model whereas row/column strips with a cutoff size of 5 have the best performance under the OpenGL cost model. This is because a cutoff size of 5 generates more patches than a cutoff size of 25 and more patches means lesser number of swaps.

5 Impact of Buffer Size

The benefits realized by using triangle strips could be further enhanced by special-purpose hardware that has additional buffer space (beyond the usual storage for two vertices) and alternate queuing disciplines. In this section, we study the impact of such resources on performance, to provide guidance for future hardware design.

Increasing the buffer size from a capacity of two vertices naturally decreases the cost of transmission, since we can now specify which of the previous k vertices in the buffer defines the next triangle. The cost of specification becomes $\lceil \lg k \rceil$ bits, instead of number of bits representing one vertex, thus enabling us to potentially represent polygonal models at a cost of less than one vertex per triangle. In our paper, we will ignore the costs of these index bits, since we only seek to determine an upper bound potential improvement in rendering time to assess whether it might be worth the increase in hardware costs.

We considered two different queuing disciplines for maintaining the buffer:

- *First-in, first-out (FIFO)* – This implies that there is no rearrangement of the vertices in the buffer, excluding swaps. FIFO is easiest to implement in hardware, and would thus be preferable if performance is comparable.
- *Least recently used (LRU)* – LRU dynamically rearranges the vertices in the buffer, by placing a vertex that was used most recently into the spot in the buffer that holds the most recently admitted vertex. The least recently used vertex is eliminated when a new vertex is added to the queue. LRU provides the benefit that popular vertices are held in the buffer in the hope that they will likely be used in the near future.

The results of running our tests on several datasets using the whole-face local triangulation method with buffer size of $k \geq 2$ are presented in Figure 10. A larger buffer size implies that we are reusing more of the vertices that were previously transmitted. These figures show the cost of the LRU and FIFO queuing disciplines versus the dataset sizes. As can be seen the advantages to be gained from larger buffer sizes diminish rapidly beyond a buffer size of about 8. For buffer sizes less than 8, LRU performs better than the FIFO scheme by a factor of about 10%.

6 Conclusions and Future Work

We have explored a total of twenty different local and global algorithms in our quest for an effective triangle strip generation algorithm that can perform well under the prevalent OpenGL cost model. Our conclusion is that the best approach for the OpenGL cost model is global row/column strips with a patch cutoff size of 5.

As can be seen from the results of Table 1, we are able to outperform the SGI algorithm significantly. We typically produce a significantly lower number of strips than they do (usually 60%-80% less strips using the local whole-triangulation algorithm), resulting in an average cost savings of about 15% less than SGI algorithm under the OpenGL model. Further, our cost averages just 10% more than the theoretical minimum of using one sequential strip with no swaps, when using the global full-patch strips algorithm with a patch cutoff size of 5, as shown in Figure 9.

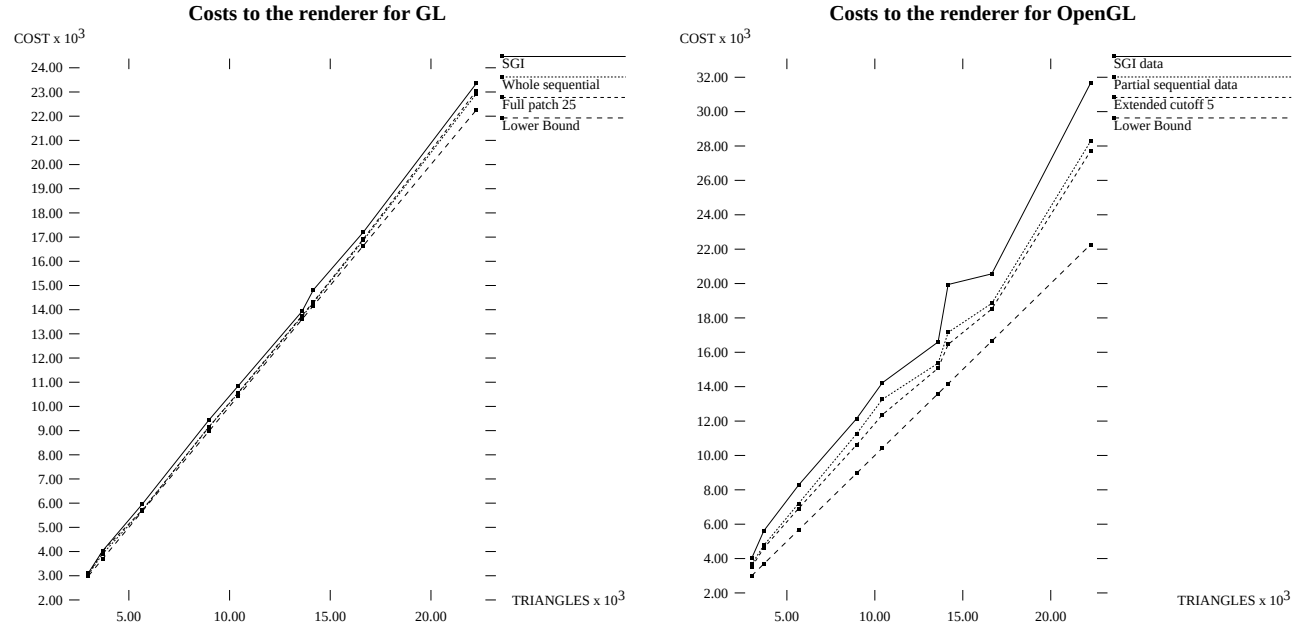
We have found that using global algorithms for detecting large strips of quads proves very effective for reducing swaps. This has proved to be quite useful for generating efficient triangle strips for the OpenGL cost model where every swap costs one vertex.

All our algorithms run in linear time. Although the SGI algorithm does have a slightly better running time, we do not believe this to be a serious drawback of our approach since the triangle-strip generation phase is typically done off-line before interactive visualization.

The results of our experiments with larger buffer sizes offer only limited room for optimism. As we increase the buffer-size the savings do increase, however the improvements diminish very quickly. LRU seems to work much better than FIFO in the smaller buffers, although this must be contrasted with the time and hardware needed to maintain a LRU buffer. The theoretical minimum of using larger buffers is the number of vertices in the model, since each vertex would only have to be transmitted exactly one time, and then could remain in the buffer forever to be used again, provided the buffer is large enough. However, in our implementation we had been assuming that the buffer gets flushed between renderings of different generalized triangle meshes, i.e. a generalized triangle mesh cannot take advantage of the buffer references left behind by a previous mesh. Even if we do not make this assumption, achieving close to the minimum requires a prohibitively large buffer, which is not feasible for hardware implementation. Further, as the result of Bar-Yehuda and Gotsman [3] shows, to achieve this minimum for a mesh of size n a buffer of size $1.649\sqrt{n}$ is necessary, thus making the size of the buffer depend on the size of the input mesh. All of these factors combined with our results seem to make a choice of a small buffer size, say around 8, attractive.

Future work includes:

- Investigate other ways to globally analyze a model prior to finding triangle strips. Currently we only find patches consisting of quadrilaterals, however we can also seek large sequential patches of other polygons, such as triangles. We can experiment with running other local options on the remaining model, although we predict that there will only be slight differences.



(a) GL Cost Model (b) OpenGL Cost Model

Figure 9: GL and OpenGL cost model comparisons.

- Creating and distributing a robust and efficient utility for creating strips for polygonal models, based on the algorithms described in this paper.
- Perform a careful study of algorithms for constructing triangle strips from fully triangulated models, since this work exploits freedom which is not present in this common situation.
- Our current cost function has been motivated by systems that are bandwidth limited or perform all transformations sequentially. However, on many multi-processor graphics systems the triangles/second curve levels-off as the system approaches its parallel processing and cache memory limits. For such a system, if most of the peak performance is achieved by strips of length, say 16 triangles, then rendering two strips of lengths 30 and 2 will be slower than rendering two strips of lengths 16 each. Our current cost model does not account for this and we plan to explore this further.

Acknowledgements

We would like to acknowledge several valuable discussions we have had on triangle strips with Joe Mitchell, Martin Held, Estie Arkin, Jarek Rossignac, Josh Mittleman, and Jim Helman. We would also like to thank the anonymous referees for their helpful comments. The datasets that we have used have been provided by Viewpoint DataLabs. Francine Evans is supported in part by a NSF Graduate Fellowship and a Northrop Grumman Fellowship. Steven Skiena is supported by ONR award 400x116yip01. Amitabh Varshney is supported in part by NSF Career Award CCR-9502239.

References

[1] K. Akeley, P. Haeberli, and D. Burns. tomesh.c : C Program on SGI Developer's Toolbox CD, 1990.

[2] E. Arkin, M. Held, J. Mitchell, and S. Skiena. Hamiltonian triangulations for fast rendering. In *Second Annual European Symposium on Algorithms*, volume 855, pages 36–47. Springer-Verlag Lecture Notes in Computer Science, 1994.

[3] R. Bar-Yehuda and C. Gotsman. Time/space tradeoffs for polygon mesh rendering. *ACM Transactions on Graphics*, 1996. (to appear).

[4] P. Bhattacharya and A. Rosenfeld. Polygonal ribbons in two and three dimensions. Technical report, Department of Computer Science, University of Maryland, 1994.

[5] J. Bose and G. Toussaint. No quadrangulation is extremely odd. Technical Report 95-03, Department of Computer Science, University of British Columbia, 1995.

[6] M. Deering. Geometry compression. *Computer Graphics Proceedings, Annual Conference Series, ACM SIGGRAPH*, pages 13–20, 1995.

[7] F. Evans, S. Skiena, and A. Varshney. Completing sequential triangulations is hard. Technical report, Dept of Computer Science, State University of New York at Stony Brook, NY 11794-4400, 1996.

[8] J. Helman. Personal Communication.

[9] H. Hoppe. Progressive meshes. *Computer Graphics Proceedings, Annual Conference Series, ACM SIGGRAPH*, 1996. (to appear).

[10] G. Narasimhan. On hamiltonian triangulations in simple polygons. In *Proceedings of the Fifth MSI-Stony Brook Workshop on Computational Geometry*, page 15, October 1995.

[11] Open GL Architecture Review Board. *OpenGL Reference Manual*. Addison-Wesley Publishing Company, Reading, MA, 1993.

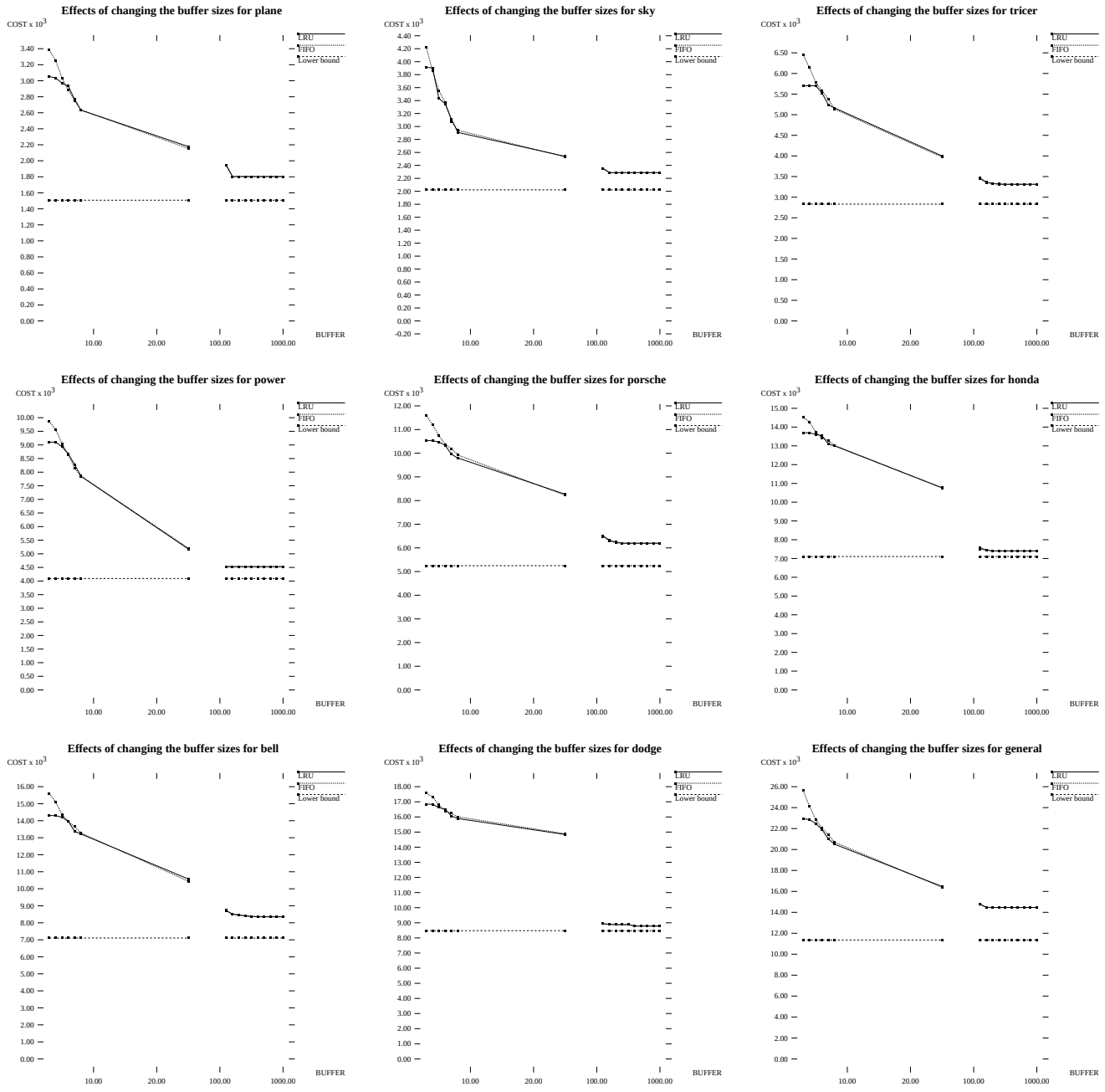


Figure 10: Cost versus buffer size for nine models.

[12] Open GL Architecture Review Board, J. Neider, T. Davis, and M. Woo. *OpenGL Programming Guide*. Addison-Wesley Publishing Company, Reading, MA, 1993.

[13] Silicon Graphics, Inc. Graphics Library Programming Guide, 1991.